Università di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e Informatica

Advanced Programming Languages

C++: class derivation

Docente:
Marco Grassia
marco.grassia@unict.it
A.A. 2025/2026

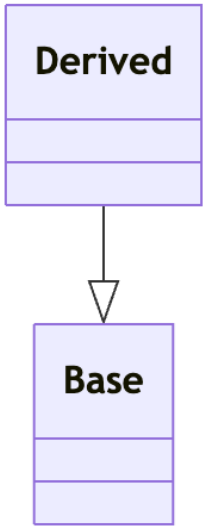
Classes: Derivation

Advanced Programming Languages: C++

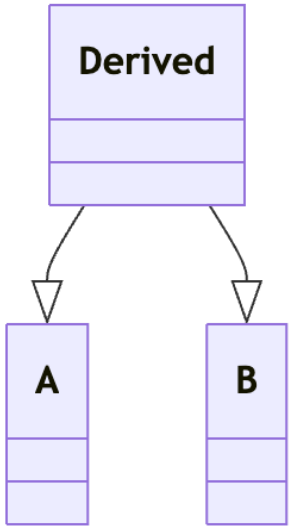
OOP: Inheritance

- **Inheritance** is a **mechanism to create a new class** (**derived**) **from an existing class** (**base**), reusing and extending its interface and implementation
- It is **meant to express hierarchical relationships**, i.e., where derived is-a base (e.g., *Circle* ***is-a*** *Shape*) and to reuse code
- In other words, inheritance **is about behavioral semantics**, not just code reuse
- Inheritance can be **single** or **multiple**
- It **does not automatically imply polymorphism**

Inheritance



Multiple Inheritance



OOP: Polymorphism

- Polymorphism is the **ability** to **treat objects of different types through a common interface and have behavior vary by the actual** (dynamic) **type**
- **Liskov Substitution Principle (LSP): derived classes should be usable wherever the base class is expected** without breaking behavior (behavioral subtyping)

```
struct Shape {
    void area();
    void perimeter();
    virtual void draw() const;
};

struct Circle : Shape {
    void radius();
    void diameter();
    void draw() const override {
        // Circle-specific drawing code
    }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

OOP: Inheritance vs polymorphism

- **Inheritance provides** the **structure for polymorphism but does not guarantee it**

- **Polymorphism requires inheritance** (or interfaces) **plus dynamic dispatch**

```
struct Shape {
    void area();
    void perimeter();
    virtual void draw() const;
};

struct Circle : Shape {
    void radius();
    void diameter();
    void draw() const override {
        // Circle-specific drawing code
    }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

C++ classes: derivation

- Derived classes can be declared with the following syntax:

  class Derived : LEVEL Base { /* … */ };

- The ***Derived*** **inherits all** ***Base*** **(parent class** or **superclass) members**
- **Inheritance access level specifier can be public, protected**, or **private, and affects how** the **members of** the **Base** class **are exposed by** the **Derived** one

```
struct Base {
   void f() {}
};
struct Derived : Base {
   void g() {}
};

Derived d;
d.f(); // inherited from Base
```

C++ classes derivation: inheritance

- A **derived class** extends a **base class** via **inheritance**
- The **derived** object **contains a base sub-object**
- Use **public inheritance** to model **“is-a”** (substitutability): a Derived **is a** Base
- Inheritance is about **behavioral semantics**, not just code reuse…
  prefer **composition** when not “is-a”
- We’ll discuss about private and protected inheritance later

```
struct Base {
   void f() {}
};
struct Derived : Base {
   void g() {}
};

Derived d;
d.f(); // inherited from Base
```

```
class Shape {
   void area();
   void perimeter();
};

class Circle : Shape {
   void radius();
   void diameter();
};
```

C++ class derivation: structural composition

- In C++, inheritance is implemented via **structural composition**:
  - Every **derived object contains the full base object** as a **sub-object,** embedded directly at the beginning of the derived object (single inheritance)
  - This is handled **automatically** by the compiler
- Every time a Derived object is created, the Base is created first

```
struct Base {
    Base() {
        println("Base Constructor");
    }
};
struct Derived : Base {
    Derived() {
        println("Derived Constructor");
    }
};

Base b;
// Prints:
//   "Base Constructor"

Derived d;
// Prints:
//   "Base Constructor"
//   "Derived Constructor"
```

C++ classes: derivation

- In C++, when **constructing a derived object, the base object is built first**. Order:
  1. **Base** classes, in declaration order
  2. **Members**, in declaration order
  3. **Derived** class body
- **Destruction reverses this order**
- **Constructors are not automatically inherited, but** since C++11 **you can use using Base::Base; to inherit (all of!) them**

```
struct Base {
    Base() {
        println("Base Constructor");
    }
};
struct Derived : Base {
    Derived() {
        println("Derived Constructor");
    }
};

Base b;
// Prints:
//   "Base Constructor"

Derived d;
// Prints:
//   "Base Constructor"
//   "Derived Constructor"
```

C++ class derivation: custom ctors and base init

- **If no base constructor is called explicitly, the default** constructor **is used (if available)**
- When **using custom constructors**, the **developer must explicitly call the appropriate constructor of the Base class**
- C++ does **not automatically forward constructor arguments to the base class**
- This behavior is crucial when the base class has no default constructor or requires specific initialization

```
struct Base {
    int a, b;

    Base(): Base(0, 0) {}
    Base(int a, int b) : a(a), b(b) {
        println("Base Constructor");
    }
};
struct Derived : Base {
    int c;

    Derived(): Derived(0, 0, 0) {}
    Derived(int a, int b, int c):
            Base(a, b), c(c) {
        println("Derived Constructor");
    }
};
```

C++ class derivation: memory layout

- **In memory layout** terms, this means that **each Derived instance stores a Base sub-object embedded its memory location**
- Note that it's **not a pointer to but the actual object**
- The **base object members can be accessed via the member named after the parent class**
  - For instance, if Derived → Base, then: derived.Base::parent_member_name

```
struct Base {
    void draw() {
        println("Base");
    }
};
struct Derived : Base {
    void draw() {
        println("Derived\n");
    }
};

Derived d;
d::draw(); // prints "Derived"
d.Base::draw(); // prints "Base"
```

C++ class derivation: structural inheritance

This type of inheritance originates from C-style structs, where aggregates are laid-out contiguously in memory. In C++, this model guarantees:

- **Substitutability: a Derived object can be used wherever a Base is expected**, because it *is* a Base
- **Performance**: **accessing members of the base class is efficient**, since **no indirection is needed to retrieve the base subobject**, whereas in languages like Java or C# inheritance is indirect and implemented via references and virtual tables
- **Correct layout** for **upcasting:** a pointer to Derived* can be safely cast to Base*, as the base subobject resides at a known offset (typically zero). This is crucial for (static) polymorphism

C++ classes derivation: inheritance

- **A derived class can choose access permissions to the public members of the parent class**
- That is, they can be kept as indicated in the parent class (public), protect or even make them private
- If you don't specify anything, then by default we mean private inheritances for classes, public for structs

Parent class	Derived class		
	public derivation	protected derivation	private derivation
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

C++ classes derivation: inheritance

- A derived class can also decide to protect the members it has inherited
  - *protected*: all public members of the parent class will become protected
  - *private*: all public members of the parent class will become private

Parent class	Derived class		
	public derivation	protected derivation	private derivation
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

Private, protected and public inheritance

- **Memory layout is identical**. In all cases, the derived object physically contains a base subobject
- Access control **only changes how the compiler enforces visibility from outside**

```
struct Base {
   int x;
};

struct Pub : public Base {};
struct Prot : protected Base {};
struct Priv : private Base {};

int main() {
   Pub pub;
   // OK: public inheritance
   // base class is accessible
   pub.x = 10;

   Prot prot;
   // prot.x = 10;    // ERROR: unaccessible
   // ERROR: base class is not accessible
   Base* b1 = &prot;

   Priv priv;
   // priv.x = 10;     // ERROR: unaccessible
   // ERROR: base class is not accessible
   Base* b2 = &priv;
}
```

Private, protected and public inheritance

- ***public Base: base subobject accessible as if public*** → *Derived* convertible to Base* externally
- **protected Base: inheritance visible only to subclasses**, not external code
- **private Base: inheritance not visible outside**; only Derived and friends can access base members

```
struct Base {
   int x;
};

struct Pub : public Base {};
struct Prot : protected Base {};
struct Priv : private Base {};

int main() {
   Pub pub;
   // OK: public inheritance
   // base class is accessible
   pub.x = 10;

   Prot prot;
   // prot.x = 10;    // ERROR: unaccessible
   // ERROR: base class is not accessible
   Base* b1 = &prot;

   Priv priv;
   // priv.x = 10;     // ERROR: unaccessible
   // ERROR: base class is not accessible
   Base* b2 = &priv;
}
```

C++ derived classes: *using* to change access level

- using **can promote inherited members to a different access level** in derived classes
- Allows inherited **members to be re-exposed** with a different access level
- **Avoids redefining the function**, preserving the original implementation while changing visibility
- Common use: make a protected base member public in a derived class

```
struct B { protected: void h(); };
struct D : B {
public:
   // h becomes public in D
   using B::h;
};
```

OOP: Composition vs Inheritance

- **Public inheritance** means that Derived **is-a Base**
- **Private inheritance** means that Derived **is implemented in terms of** Base but **is-not-a** Base
  - **This breaks substitutability** (LSP) as Derived cannot be used where Base is expected
  - **Base and Derived are tightly coupled** (e.g., as Car depends on Engine's internal details) making it harder to change, test or maintain

```
struct Engine {
   void start();
};
struct Car : private Engine {
   void drive() {
      start(); // OK: Car reuses
   }       // Engine's implementation
};

Car c;
// Engine* e = &c;
// ERROR: Car is-not-an Engine
```

```
struct Engine {
   void start();
};
struct Car {
   // Car has-an Engine
   Engine engine;
   void drive() {
      engine.start();
   }
};
```

OOP: Composition vs Inheritance

- In this case, prefer Composition, i.e., a clear **“has-a” relationship**
  - This **provides better encapsulation and decoupling** (e.g., Engine can evolve independently from Car)

- **Protected inheritance is rare**; **it means “is-a for derived classes only**.” External code cannot treat Derived as Base. **Used** mainly **for framework integration** or partial interface exposure

```
struct Engine {
   void start();
};
struct Car : private Engine {
   void drive() {
      start(); // OK: Car reuses
   }       // Engine's implementation
};

Car c;
// Engine* e = &c;
// ERROR: Car is-not-an Engine
```

```
struct Engine {
   void start();
};
struct Car {
   // Car has-an Engine
   Engine engine;
   void drive() {
      engine.start();
   }
};
```

OOP: Composition vs Inheritance

- **Private inheritance** is **rarely needed** and **often** used **only for code reuse**, which is a **design smell**
- Use **composition** when the relationship is **“has-a”**
- Use **public inheritance** only for **true “is-a”**
- Avoid **private inheritance** for mere code reuse, prefer composition that is almost always better

```
struct Engine {
   void start();
};
struct Car : private Engine {
   void drive() {
      start(); // OK: Car reuses
   }       // Engine's implementation
};

Car c;
// Engine* e = &c;
// ERROR: Car is-not-an Engine
```

```
struct Engine {
   void start();
};
struct Car {
   // Car has-an Engine
   Engine engine;
   void drive() {
      engine.start();
   }
};
```

C++ class derivation: shadowing / name hiding

- When a **derived class declares** a **member with** the **same name as one in the base class, the base member is hidden in the scope of the derived class**
- The base member still exists in memory, but it is “hidden” and **not directly accessible** without **explicit qualification** (i.e., Base::x)
- Avoid name hiding unless necessary, as it can lead to confusing bugs
- Consider using **composition** instead of inheritance **when name conflicts are likely**

```
struct Base {
   int x;
};

struct Derived : Base {
   int x;
};

Derived d;

println("{} != {}",
            (void*)&d.Base::x,
            (void*)&d.x);
// Prints two different addresses
// e.g.:
// 0x16d95edc0 != 0x16d95edc4
```

C++ classes: static vs dynamic binding

- By default, **function calls** are **resolved at compile time** (**early or static binding**)
- Binding is association of a symbol within a program with an address in memory
- If a **base pointer refers to a derived object, functions do not dispatch dynamically by default** and will call the base's member instead
- We **need dynamic** (**a.k.a. late / runtime**) **binding to achieve dynamic polymorphism**

```
struct Base {
    void draw() {
        std::println("Base");
    }
};
struct Derived : Base {
    void draw() {
        println("Derived\n");
    }
};
Base* bp = new Derived;
bp->draw();
// prints "Base" (static/early binding)
```

Compiler binds calls based on **static type** (Base*), not dynamic type (Derived*)

https://stackoverflow.com/questions/18035293/what-is-early-static-and-late-dynamic-binding-in-c

OOP: static vs dynamic polymorphism

- *"**Static**, compile time **binding is easy**. There's **no polymorphism** involved. **You know the type** of the object **when you write** and compile and run the code. Sometimes a dog is just a dog.*
- *Dynamic, **runtime binding is where polymorphism comes from**. If you have a **reference that's of parent type at compile type, you can assign a child type to it at runtime**. The behavior of the reference will magically change to the appropriate type at runtime. [In C++] A **virtual table lookup will be done to let the runtime figure out what the dynamic type is.**"*

```
struct Shape {
   void area();
   void perimeter();
   virtual void draw() const;
};

struct Circle : Shape {
   void radius();
   void diameter();
   void draw() const override {
      // Circle-specific drawing code
   }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

C++ derivates: *virtual* and late binding

- In C++, **late binding** (dynamic dispatch) is **enabled by using** the ***virtual*** keyword in the **function declaration** (i.e., in the base class, which makes it polymorphic)
- **Late binding applies only when accessed through a base class pointer or reference**
- This is the **only case in C++** where **runtime type checking** is performed to resolve function calls
- **Without *virtual* it's just name hiding**

```
struct Shape {
    void area();
    void perimeter();
    virtual void draw() const;
};

struct Circle : Shape {
    void radius();
    void diameter();
    void draw() const override {
        // Circle-specific drawing code
    }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

C++ derivates: *override*

- The ***override*** keyword is a (**optional**) specifier used to **explicitly mark** that **a member function in a derived** class **is meant to override** a **virtual** function **from a base** class
- It **ensures that the function actually overrides a virtual** function from the base class. **If there's a mismatch** (e.g., non-virtual member, wrong signature or spelling), **the compiler will generate an error**
- It improves **code safety** and **readability**, especially in large codebases

```
struct Shape {
    void area();
    void perimeter();
    virtual void draw() const;
};

struct Circle : Shape {
    void radius();
    void diameter();
    void draw() const override {
        // Circle-specific drawing code
    }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

C++ dynamic polymorphism: comparison

- In **C++**, **virtual** dispatch **must be explicitly enabled to avoid general** unnecessary **runtime overhead**. The *override* is optional for overriding virtual members, but highly recommended
- **Only** the **declaration** of the function **needs to be marked with virtual**; **derived classes inherit** this behavior **automatically**, even if not explicitly marked
- In **Java**, all methods are virtual by default
- C# has a similar behaviour to C++, but *override* is **mandatory**

```
struct Shape {
    void area();
    void perimeter();
    virtual void draw() const;
};

struct Circle : Shape {
    void radius();
    void diameter();
    void draw() const override {
        // Circle-specific drawing code
    }
};

Shape* s = new Circle();
s->draw();
// calls Circle::draw() at runtime
```

C++ polymorphism: recap on static vs dynamic poly.

- **Compile-time (static)**: Function overloading, templates, …
- **Run-time (dynamic)**: Achieved via **virtual functions** and **pointers/references to base. It requires:**
  - A **polymorphic base class** (with at least one virtual function)
  - Access through **base pointer or reference**
  - **Assigning to a Base class object would lead to object slicing**

```
class Base{
public:
virtual int f() const { return 1; }
};

class Derived: public Base{
public:
int f() const { return 2; }
};
```

```
Derived d;
 Base* b1 = &d;
 Base&b2 = d;
 Base b3;

 // Late binding:
 cout << "b1->f() = " << b1->f() << endl;
 cout << "b2.f() = " << b2.f() << endl;

 // Early binding
 cout << "b3.f() = " << b3.f() << endl;
```

C++ dynamic polymorphism: *vtables*

- A **vtable** is a hidden **table of member function pointers** generated by the compiler for each class that has at least one **virtual function member**
- **There is one vtable per class** (not per object) that **maps virtual functions to the correct implementation** depending on the object's *dynamic type*
- **Each object stores a** hidden **pointer** (often called **vptr**, vpointer) **that points to the vtable of its dynamic type**
- Overridden methods replace the base entries in the vtable

```
Object of Derived:
[ vptr → vtable_of_Derived |
  Base part |
  Derived members
]

vtable_of_Base:
  f → Base::f

vtable_of_Derived:
  f → Derived::f
```

C++ dynamic polymorphism: *vtables*

- When a virtual function is called via a **base class pointer or reference**, the program:
  1. Retrieves the vtable address via the vptr
  2. Looks up the correct function pointer in the vtable
  3. Invokes the function corresponding to the object's **runtime type**
- This mechanism enables **dynamic dispatch**, but introduces **runtime overhead** and memory **layout complexity**

```
Object of Derived:
[ vptr → vtable_of_Derived |
  Base part |
  Derived members
]

vtable_of_Base:
  f → Base::f

vtable_of_Derived:
  f → Derived::f
```

C++ polymorphism: what about data members?

- C++ supports **virtual functions**, **not** virtual **data members**
- Virtual dispatch works via **vtable** for functions
- Data members are accessed by **fixed offsets and cannot vary at runtime**
- You **cannot override** a **field** in a derived class **polymorphically…**

```
struct Base {
   unsigned x = 0;

   virtual void printX() {
      println("Base x: {}", x);
   }
};

class Derived : public Base {
   unsigned x;
   void printX() override {
      println("Base x: {}", this->Base::x);
      println("Derived x: {}", this->x);
   }
};

Base* a = new Derived();

a->x = 10;
a->printX();
```

```
Base x: 10
Derived x: 0
```

C++ polymorphism: what about data members?

- … **but** you **can expose it** through **virtual accessors**
- **Instead** of **making the attribute virtual**: **keep it private** or protected **and provide virtual getter/setter** or accessors (refs)

```
class BaseGetSet {
protected:
   unsigned base_x = 0;
public:
   virtual unsigned getX() const { return base_x; }
   virtual void setX(unsigned v) { base_x = v; }

   virtual void printX() const {
      std::cout << "BaseGetSet x: " << getX() << "\n";
   }

   virtual ~BaseGetSet() = default;
};

class DerivedGetSet : BaseGetSet {
   unsigned derived_x = 0;
public:
   unsigned getX() const override { return derived_x; }
   void setX(unsigned v) override { derived_x = v; }

   void printX() const override {
      std::cout << "DerivedGetSet x: " << getX() << "\n";
   }
};
```

C++ polymorphism: what about data members?

- … **but** you **can expose it** through **virtual accessors**
- **Instead** of **making the attribute virtual**: **keep it private** or protected **and provide** virtual getter/setter or **accessors (refs)**

```cpp
struct BaseAccessor {
protected:
    unsigned base_x = 0; // storage di default per la base
public:
    virtual unsigned& x() { return base_x; }
    virtual const unsigned& x() const { return base_x; }

    virtual void printX() const {
        std::cout << "BaseAccessor x: " << x() << "\n";
    }

    virtual ~BaseAccessor() = default;
};

struct DerivedAccessor : BaseAccessor {
    unsigned derived_x = 0; // storage per la derived

    unsigned& x() override { return derived_x; }
    const unsigned& x() const override { return derived_x; }

    void printX() const override {
        std::cout << "DerivedAccessor x: " << x() << "\n";
    }
};
```

C++ derivates: *final* (C++11)

- The ***final*** keyword is **used to prevent**:
  - **further overriding of a virtual function**
  - **inheritance from a class**
- Use final to **enforce design decisions** and avoid accidental overrides
- **Helps the compiler optimize** virtual calls **via de-virtualization**. This allows inlining and other optimizations

```
struct Base {
   virtual void speak() final;
   // Cannot be overridden
};

struct Derived : Base {
   void speak() override;
   // ❌ Error: 'speak' is final in Base
};
```

```
struct FinalClass final {
   void greet() {}
};

struct SubClass : FinalClass {};
// ❌ Error: cannot inherit from a final class
```

C++:
abstract classes

- An **abstract class** is a class that contains at least one **pure virtual function**, serving as an **interface**
- Abstract classes **cannot be instantiated directly,** as they are **meant to be inherited** by other classes that **must implement all pure virtual functions to become concrete** and instantiable
- The *member function = 0* **syntax marks a member function as pure virtual**, indicating it has no implementation in the base class
- A **pure virtual function remains virtual in all derived classes**, even if not explicitly marked
- Constructors / destructors cannot be pure virtual

```
class Shape {
public:
    // Pure virtual function
    virtual void draw() = 0;
};
class Circle : public Shape {
public:
    void draw() override {
        println("Drawing a"
                "circle");
    }
};
```

C++ derivates: virtual destructor

- In C++, when **deleting an object through** a **base class pointer**, the destructor of the **derived class** is called **only if the base** class **destructor is virtual**
- Without a virtual destructor, only the base class destructor runs, **potentially causing resource leaks** or **incomplete cleanup**

```
struct Base {
   ~Base() { std::cout << "Base destroyed\n"; }
};
struct Derived : Base {
   ~Derived() { std::cout << "Derived destroyed\n"; }
};

Base* b = new Derived;
delete b; // Output: "Base destroyed"

Derived* d = new Derived;
delete d; // Output: "Derived destroyed" "Base destroyed"
```

C++ derivates: virtual destructor

- Always declare destructors as virtual in **polymorphic base classes**
- This ensures **correct destruction** of derived objects
- **Avoid virtual** destructors **in non-polymorphic or final** classes for performance

```
struct Base {
    virtual ~Base() { std::cout << "Base destroyed\n"; }
};
struct Derived : Base {
    ~Derived() { std::cout << "Derived destroyed\n"; }
};

Base* b = new Derived;
delete b; // Output: "Derived destroyed" "Base destroyed" ✅

Derived* d = new Derived;
delete d; // Output: "Derived destroyed" "Base destroyed" ✅
```

C++ classes: destructor are called automatically when…

1. **a local-object (stack-allocated) goes out of scope**
2. ***delete* is used on a pointer to a heap-allocated object**
3. **a temporary object is destroyed.** For example, after an expression ends:…
4. **a base class or member object is destroyed.** In inheritance or composition, destructors are called in reverse order of construction
5. **a container (e.g., std::vector) holding objects is destroyed**, the destructor of each element is called automatically

```
struct A {
   A() { println("A's constructor"); }
   ~A() { println("A's destructor"); }
};
struct B : A {
   B() { println("B's constructor"); }
   ~B() { println("B's destructor"); }
};
struct C : B {
   C() { println("C's constructor "); }
   ~C() { println("C's destructor"); }
};
struct D {
   C c;
   D () { println("D's constructor"); }
   ~D() { println("D's destructor"); }
};
int main() {
   D d{};
}
```

```
A's constructor
B's constructor
C's constructor
D's constructor
D's destructor
C's destructor
B's destructor
A's destructor
```

C++ classes: recap on polymorphism

- A **polymorphic base has at least a virtual member function**
- **Virtual dispatch** / dynamic polymorphism **needs virtual members AND a ref/pointer to the object**. It **does not work by value**
- **Runtime** member **function** is **found in** the **class vtable referenced by** the vpointer (**vptr**) **in the object**
- Remember to **use a virtual destructor** to avoid leaks when deleting via base pointer

```
class Shape {
   Point center;
public:
   explicit Shape(Point c) : center(c) {/* ... */}
   // Pure virtual destructor and member function(s)
   virtual ~Shape() { println("~Shape"); /* cleanup code */ };
   virtual void draw() = 0;
};

class Circle : public Shape {
   double radius;
public:
   Circle(Point c, double r) : Shape(c), radius(r) {/* ... */}
   ~Circle() override { println("~Circle"); /* cleanup code */ }
   void draw() override { println("Drawing a circle"); /*... */ }
};

Shape*c = new Circle(Point(0, 0), 5.0);
c->draw(); // Output: "Drawing a circle"
delete c; // Calls Circle's and then Shape's destructors

// Output:
Drawing a circle
~Circle
~Shape
```

C++ classes: recap on polymorphism

- **Pure virtual classes are abstract**
- Use ***override*** **to get compile-time checks;** ***final*** **to stop further overrides**
- **Overloads** in Derived **can hide base overloads; re-expose with using Base::member_f_name**
- **Virtual destructors in Base ensure the Derived destructor is called using Base pointers** (Base*)

```
class Shape {
   Point center;
public:
   explicit Shape(Point c) : center(c) {/* ... */}
   // Pure virtual destructor and member function(s)
   virtual ~Shape() { println("~Shape"); /* cleanup code */ };
   virtual void draw() = 0;
};

class Circle : public Shape {
   double radius;
public:
   Circle(Point c, double r) : Shape(c), radius(r) {/* ... */}
   ~Circle() override { println("~Circle"); /* cleanup code */ }
   void draw() override { println("Drawing a circle"); /*... */ }
};

Shape*c = new Circle(Point(0, 0), 5.0);
c->draw(); // Output: "Drawing a circle"
delete c; // Calls Circle's and then Shape's destructors

// Output:
Drawing a circle
~Circle
~Shape
```

Pure virtual vs declaration only

Aspect	Pure virtual (=0)	Declaration only (no body yet)
Instantiability	Makes the class **abstract** until overridden with a non-pure in a derived class	Class remains **concrete** (assuming no other pure virtuals)
Definition required?	Optional (except for **pure virtual destructor**, which **must** be defined); providing one does **not** make it non-pure	**Required** if the function is used; otherwise link error
Purpose	Specify an **interface contract** to be implemented by derived classes	Ordinary function/virtual meant to have an implementation provided elsewhere
Overriding in derived	**Mandatory** (to make a concrete type) unless the derived also keeps it pure	**Optional**; derived may override, but base already supplies behavior (once defined)
Calls in ctor/dtor via virtual dispatch	**UB** if the dynamic dispatch reaches a pure virtual; call via Base::f() only if a definition exists	Normal rules apply; virtual dispatch calls the most-derived non-pure override available for the current construction stage

C++:
Run-Time Type Information (RTTI)

- **Run-Time Type Information (RTTI)** is a **mechanism** that allows the program **to determine** the **actual type of an object at runtime**, e.g., when accessed through a base class pointer or reference
- It **only works with polymorphic types**, i.e., classes with at least one virtual function, as type info are stored into the vtable and can be disabled (e.g., via compiler flags)

Feature	C++ RTTI	Java / C# Type Info
Activation	Requires virtual functions	Always available
Type info	typeid	getClass() / GetType()
Downcasting	dynamic_cast	instanceof / as
Performance	Slight overhead	Built-in, optimized

C++ RTTI: *typeid* operator

- typeid **operator returns a** type_info object **describing** the **actual type** of an expression, useful for type comparisons
- type_info main members are:
  - **const char* name() const,** implementation-defined string with the type name (may be mangled)
  - **bool operators** == **and** != to **compare type** identities

```
struct Base { virtual ~Base() {} };
struct Derived : Base {};

Base* b = new Derived();

// static type
println("{}", typeid(b).name());
// → "Base*"

// dynamic type (polymorphic)
println("{}", typeid(*b).name());
// → "Derived"

println("Base*? {}", typeid(b) == typeid(Base*));
// → Base*? true
println("Derived*? {}", typeid(b) == typeid(Derived*));
// → Derived*? false
println("Derived? {}", typeid(b) == typeid(Derived));
// → Derived? false
```

C++ RTTI: dynamic_cast operator

- dynamic_cast **operator <u>safely</u> casts** a base class pointer/reference **to a derived class** (downcasting)
- **If the cast is invalid,** it returns ***nullptr* if the cast is invalid for pointers** or **throws** *std::**bad_cast*** **for references** (that cannot be null)
- Avoid unsafe C-style or reinterpret_cast

```
struct Base { virtual ~Base() {} };
struct Derived : Base {};

Base* b = new Derived();

auto* d = dynamic_cast<Derived*>(b);
if (d != nullptr) {
    std::cout << "Pointer cast succeeded\n";
}

try {
    auto& d_ref = dynamic_cast<Derived&>(*b);
    std::cout << "Reference cast succeeded\n";
} catch (const std::bad_cast& e) {
    std::cout << "Reference cast failed: " << e.what() <<
"\n";
}
// → Pointer cast succeeded
// → Reference cast succeeded
```

C++ derivates: conversions

- **Object slicing = assigning a Derived to a Base by value** (even in parameter passing) **copies only the Base subobject**, causing information loss, as the “extra” part is **discarded**
- **Upcast**: converting a Derived* → Base* (or reference)
  - **Always safe, implicit** as there is no data loss since the Base subobject is real; **no runtime cost**
  - static_cast or even dynamic not needed

```
// Derived part is sliced off
Base b = Derived();
// b is a Base object, not a Derived object
println("{}", b.some_derived_method());
// Error: no such method in Base
```

```
Derived* derived = new Derived();
// Safe, implicit upcasts
Base* upcast_derived = derived;

Derived &derived_ref = *derived;
Base &upcast_derived_ref = *derived;
```

C++ derivates: conversions

- **Downcast: converting a** Base* → Derived*
  - Potentially unsafe
  - Use a dynamic_cast<> that is checked at runtime
  - **Never use unsafe C-style** or reinterpret_cast

```
Base* base_ptr = new Derived();

// Safe if Base is polymorphic
downcast_base = dynamic_cast<Derived*>(base_ptr);
if (!downcast_base) {
   println("Invalid cast");
}
// Unsafe if base_ref is not actually a Derived
Derived& base_ref = *base_ptr;
try {
   Derived& der_ref = dynamic_cast<Derived&>(base_ref);
} catch (const std::bad_cast& e) {
   println("Invalid cast: {}", e.what());
}

// DANGER ZONE
Derived* unsafe_ptr = reinterpret_cast<Derived*>(base_ptr);
Derived* unsafe_ptr2 = (Derived*)base_ptr;
// Use with extreme caution (usually for low-level code) or
// avoid completely.
```

C++ classes: recap on polymorphism

- **Never store polymorphic objects by value**, that would **lead to slicing**
- **Upcasts are safe:** Derived are always Base, including their membes)…
- **Downcasts are not: use RTTI** through dynamic_cast<…> when needed

```
struct A { virtual ~A() = default; };
struct D : A { /* whatever */ };

// A is NOT a pointer but the actual object
// as a consequence, the D object is sliced
// and the non-A part just removed
A parent_var = D();
```

```
struct A { virtual ~A() = default; };
struct B { virtual ~B() = default; };
struct D : A, B {};

A* pa = new D;
// sibling cast = casting between two base classes of
// the same most-derived object
// ⚠ This compiles only if you force it, but it's unsafe:
B* pb = static_cast<B*>(pa); // ❌ causes UB

B* pb = dynamic_cast<B*>(pa); // ✅ Safe
```

OOP: Composition vs Inheritance

- **Private inheritance** is **rarely needed** and **often** used **only for code reuse**, which is a **design smell**
- Use **composition** when the relationship is **“has-a”**
- Use **public inheritance** only for **true “is-a”**
- Avoid **private inheritance** for mere code reuse, prefer composition that is almost always better
- **Exception: interfaces** (pure abstract bases) are fine for multiple inheritance; when you need to **override virtual functions or access protected members for framework integration**

```
struct Engine {
   void start();
};
struct Car : private Engine {
   void drive() {
      start(); // OK: Car reuses
   }       // Engine's implementation
};

Car c;
// Engine* e = &c;
// ERROR: Car is-not-an Engine
```

```
struct Engine {
   void start();
};
struct Car {
   // Car has-an Engine
   Engine engine;
   void drive() {
      engine.start();
   }
};
```

C++ classes: Multiple inheritance

Advanced Programming Languages: C++

OOP: multiple inheritance

- Multiple inheritance **allows a class to inherit from more than one base class**
- **Typical use cases include:**
  - **Combining reusable components with distinct responsibilities,** useful – for instance – for composing **orthogonal behaviors** (e.g., logging + serialization)
  - **Implementing interfaces alongside a concrete base class**

```
struct Logger {
   void log(const std::string& msg) const;
};
struct Serializable {
   virtual std::string serialize() const = 0;
};

class Data : public Logger, public Serializable {
   std::string serialize() const override {
      log("Serializing Data object");
      return "data";
   }
};
```

C++ classes: multiple inheritance

- **C++** is **one of the few** (mainstream) languages **that supports multiple inheritance of implementation**, not just interfaces
- It works by **composing** (combining) **multiple base sub-objects** physically
- A derived class **lists multiple bases separated by commas**; **access specifiers** (public, protected, private) **apply per base**
- For instance, with **public inheritance**, C is-a A and is-a B

```
class A { void fa(); };
class B { void fb(); };

class C : public A, public B {
   void fc();
};

int main() {
   C c;
   c.fa(); // from A
   c.fb(); // from B
   c.fc(); // from C
}
```

OOP multiple inheritance: Comparison with other languages

- **Java / C#**: both support **single inheritance of classes** and **multiple interfaces**
- **Go** does **not support inheritance at all**; it uses composition via struct embedding and interfaces for polymorphism
- **Python supports multiple inheritance** with method resolution order (MRO)**.** In multiple inheritances, the methods are executed based on the order specified while inheriting the classes

C++ classes: multiple inheritance

- While powerful, multiple inheritance **introduces complexities** such as:
  - **Ambiguities** when two bases define the same member
  - The **Diamond Problem** and duplicated base subobjects
  - **Complex object layouts** and pointer adjustments
- It is not inherently bad, but it requires **careful design and deep understanding**

```
class A { void fa(); };
class B { void fb(); };

class C : public A, public B {
   void fc();
};

int main() {
   C c;
   c.fa(); // from A
   c.fb(); // from B
   c.fc(); // from C
}
```

C++ multiple inheritance: ambiguities

- **If two base** classes **define the same member's name**, the **compiler cannot decide which one** to call
- **Solutions:**
  1. **Qualify the call:** specify to which base you're referring to, e.g., obj.A::f()
  2. ***Using*-declaration** to expose one base's overloads
  3. **Override in derived class** to unify behavior

```
class A { public: void f(); };
class B { public: void f(); };
class C : public A, public B { };

C c;
// c.f(); // error: ambiguous
c.A::f(); // OK ( or c.B::f() )

// 2
class C : public A, public B  {
   // E.g., resolve ambiguity in favor of A's
   using A::f;
};

// 3
class C : public A, public B  {
   void f() {
      A::f(); // choose A's f()
      // ... or define a new f()
   }
};
```

C++ multiple inheritance: memory layout

- Multiple inheritance is **implemented via derivates and is structural**: **every base** class **becomes** a real **sub-object**
- In other words, a Derived object contains **one sub-object per base class**
- **Each base keeps its** own **members and** (if virtual methods exist) **its own *vptr***
- For instance, the memory layout of the example on the right would look like:

```
[ A-subobject    | B-subobject    |D-members ]
[ vptr_A         | vptr_B         | ...      ]
```

```
class A { virtual void f(); };
class B { virtual void g(); };

class D : public A, public B {
   void f() override;
   void g() override;
};
```

C++ multiple inheritance: polymorphism, VTables and pointer adjustments

- With MI, an object may contain **one vptr per polymorphic base sub-object** (each base with virtual functions has its own vtable pointer)
- When calling a virtual method through a base pointer (e.g., B*), the compiler may need to **adjust the *this* pointer** if the actual method implementation resides in a derived class (D)

```
class A { virtual void f(); };
class B { virtual void g(); };

class D : public A, public B {
   void f() override;
   void g() override;
};

A* a = new D;

// ❌ Undefined behavior
B* b1 = static_cast<B*>(a);
// ✅ Safe, returns correct subobject
B* b2 = dynamic_cast<B*>(a);
```

C++ multiple inheritance: polymorphism, VTables and pointer adjustments

- static_cast only works when the conversion is **known at compile time** (e.g., upcast or downcast within a single inheritance chain)
- **In multiple inheritance, the relationship between types may not be linear** and compiler cannot guarantee correctness

```
class A { virtual void f(); };
class B { virtual void g(); };

class D : public A, public B {
    void f() override;
    void g() override;
};

A* a = new D;

// ❌ Undefined behavior
B* b1 = static_cast<B*>(a);
// ✅ Safe, returns correct subobject
B* b2 = dynamic_cast<B*>(a);
```

C++ multiple inheritance: polymorphism, VTables and pointer adjustments

- dynamic_cast **is necessary**, as it performs **runtime type checking using RTTI** (Run-Time Type Information)
- It **safely handles cross-casts** (e.g., from A* to B* when both are bases of D) **and adjusts the** pointer **correctly to the target subobject**
- However, **it requires polymorphic types** (at least one virtual function in the hierarchy) for RTTI and, for this, **has runtime performance impact**

```
class A { virtual void f(); };
class B { virtual void g(); };

class D : public A, public B {
    void f() override;
    void g() override;
};

A* a = new D;

// ❌ Undefined behavior
B* b1 = static_cast<B*>(a);
// ✅ Safe, returns correct subobject
B* b2 = dynamic_cast<B*>(a);
```

C++ MI practical patterns: Interface Segregation via Abstract Bases

- **Avoid deep or complex hierarchies** unless strictly necessary
- **Prefer multiple inheritance of pure abstract interfaces** (classes with only pure virtual functions and no data members)
- Combine **at most one concrete base class** with such interfaces
- This approach **minimizes ambiguity**, prevents **duplicate state**, and simplifies **object layout and constructor order**

```
struct IDrawable { // Interface 1
   virtual ~IDrawable() = default;
   virtual void draw() const = 0;
};
struct ISerializable { // Interface 2
   virtual ~ISerializable() = default;
   virtual std::string serialize() const = 0;
};
class ShapeBase { // Common base class
   protected: Point center;
};

struct Triangle : ShapeBase, IDrawable, ISerializable {
   void draw() const override { /*...*/   }
   std::string serialize() const override {
      return "Triangle";
   }
};
```

OOP: *mixins*

- A **mixin** is a class that can be "**mix**-ed **in**" to provide **additional behavior** to other classes through **inheritance**, **without representing an "is-a" relationship** in the classical sense
- Mixins are **meant for code reuse**, not for defining complete types
- They are typically **small, focused units of** (orthogonal) **functionality**
- **Each** mixin **adds one capability**, **independently of others** (e.g., *logging*, *serialization*, *comparison*, *event handling*)
- A class can **inherit from multiple mixins** to compose behaviors at compile-time or runtime

OOP: mixins characteristics

- They usually have **no** (**or minimal**) **internal state** to avoid ambiguity and shared ownership issues
- **Composable by inheritance:** the final derived class combines behaviors
- **Often language- or framework-specific:**
  - Python and Ruby support *mixin modules* natively
  - C++ typically implements mixins via **templates** (often using **CRTP**)
  - Java achieves similar effects via **interfaces with default methods**
  - Go uses composition and interface-based mixins

C++ MI practical patterns: Mixins / Policy Classes (Templates)

- Mixins in C++ are a **design technique** that uses **multiple inheritance (and often templates) to add reusable functionality** to classes without creating deep, rigid hierarchies
- It can just **provide functionality to derived classes, possibly through virtual inheritance or templates** that don't depend on Derived

```
class LoggingMixin { // Self-contained behavior
public:
    void log(const std::string& msg) const {
        std::cout << "[LOG] " << msg << '\n';
    }
};

class Component : public LoggingMixin {
public:
    void run() {
        // Mixin-inherited behavior
        log("Running component");
    }
};
```

C++ MI practical patterns: Mixins / Policy Classes (Templates)

- Unlike traditional base classes, **mixins** are **usually**:
  - **Not meant to be used alone** (they are often incomplete by themselves)
  - **Stateless or lightweight**, often implemented as templates
  - Designed to **compose behaviors at compile time**
- Unlike inheriting multiple interfaces, **they provide implementation!**

```
template<class T>
struct Logging {
   void log(const char* m){ /*...*/ }
};
template<class T> struct RefCounted {
   void add_ref(){ /*...*/ }
};

struct Widget : Logging<Widget>, RefCounted<Widget> {
   /*...*/
};
```

C++ MI mixins: Why use them?

- **Avoid code duplication** by reusing small, focused behaviors
- **Encourage modular design** and **behavioral composition**
- **Provide flexible composition** without forcing a single inheritance chain
- Keep **runtime overhead close to zero** (because they are often templates, resolved at compile time)
- **Avoid** the rigidity of **deep inheritance hierarchies**
- **Enable reusable policies or traits** shared across unrelated classes

C++ MI practical patterns: “Curiously Recurring Template Pattern” (CRTP)

- A **template pattern where** a **class inherits from a template parameterized with itself**
- It is **meant** to **enable static (compile-time) polymorphism via composition**
- A **base template uses** the **derived type to call implementation hooks** without vtables, but a similar *shape* to dynamic polymorphism
- Used widely in the standard library and modern libraries

```
template<class Derived>
struct Comparable {
   friend bool operator==(
      const Derived& a,
      const Derived& b
      ) {
      return a.compare(b)==0;
   }
   // ...
};
struct Point : Comparable<Point> {
   int compare(const Point& other) const {
      /*...*/ return 0;
   }
};
```

C++ MI practical patterns: “Curiously Recurring Template Pattern” (CRTP)

- **No runtime overhead** (no vtable)
- **Can be combined with other CRTP mixins** to stack behaviors
- The **base class can call methods of the derived class via static cast** (if T not available)
- Requires the derived class to implement specific methods
- “Interface” is enforced at **compile time**

```
template<typename Derived>
struct Printable {
   void print() const {
      // Static cast of 'this' to Derived type
      std::cout <<
         static_cast<const Derived*>(this)->toString()
         << std::endl;
   }
};
struct Point: Printable<Point> {
   std::string toString() const { return "(x, y)"; }
};

// Usage:
Point p;
// Calls Point::toString() via Printable mixin
p.print(); // with static polymorphism (no virtual)
```

C++ multiple inheritance: practical patterns

Example of CRTP

The two mixins are parametrized by the actual type

```cpp
// A mixin for logging
template<typename T>
struct Logging {
   void log(const std::string &msg) {
      std::cout << "[" << typeid(T).name() << "] " << msg << "\n";
   }
};

// A mixin for reference counting
template<typename T>
struct RefCounted {
   void add_ref() { ++ref_count; }
   void release() { if (--ref_count == 0) delete static_cast<T*>(this); }
private:
   int ref_count = 0;
};

// Combine behaviors in a concrete class
struct Widget : Logging<Widget>, RefCounted<Widget> {
   void doWork() { log("Working..."); }
};

Widget w;
w.doWork();
w.add_ref();
w.release();
```

Multiple Inheritance: Interfaces Inheritance vs Mixins vs CRTP

Aspect	Interface Inheritance	Mixins	CRTP
Polymorphism	Runtime (via virtual)	Optional	Compile-time (static)
Overhead	Vtable for interface	Minimal / none	None
Base Content	Only pure virtual functions	Concrete implementations	Template with static cast
Typical Use	Contracts / APIs	Behavior reuse	Performance optimization
Type Erasure	Yes (via base pointers)	Limited	No

Type erasure = runtime polymorphism via base pointers/references. The actual derived type is "erased" at runtime; you only see the interface. Example:
std::vector<Drawable*> shapes; *// works because Drawable is a base interface*

C++ multiple inheritance: recap on when to use it

- **Good fits:**
  - **Aggregate multiple interfaces** (pure abstract classes)
  - **Combine orthogonal behaviors** via **mixins/policies/CRTP**
  - **Avoid duplicated base state in a diamond** (use virtual inheritance)
- **Avoid:**
  - **Multiple concrete** implementation bases with shared mutable state
  - Deep, brittle hierarchies, prefer **composition**
- A pragmatic rule: *At most one concrete base; others should be interfaces*

C++ multiple inheritance: casting rules cheat sheet

- **Upcast** to a specific base is **unambiguous if unique base** sub-object (**i.e., virtual** base)
- Remember that **base would be ambiguous in non-virtual diamond** unless you qualify/cast explicitly to which subobject you're referring to (e.g., c.A::name)

```
// polymorphic base
struct A {
   virtual ~A() = default;
};

// virtual base (unique A)
struct B : virtual A {};

struct C : virtual A {};

struct D : B, C {};

D d;
// ✅ upcast is unambiguous with virtual base
A *pa = &d;
// ✅ ordinary upcast
B *pb = &d;
// Without virtual inheritance, upcasting to
//   A may be ambiguous
```

C++ multiple inheritance: casting rules cheat sheet

- **Cross-cast** A* → B* within same object (i.e.,pointer to one base (A*) and want to cast to another base (B*) of the **same most-derived object**)
- Requires **polymorphic** base and dynamic_cast
- **Uses RTTI** to find the correct subobject offset

```
struct A { virtual ~A() = default; };
struct B { virtual ~B() = default; };
struct D : A, B {};

A* pa = new D;

// ✅ OK: polymorphic hierarchy + RTTI
B* pb1 = dynamic_cast<B*>(pa);

// ⚠ UB if layout differs (don't do this)
B* pb2 = static_cast<B*>(pa);
```

C++ multiple inheritance: casting rules cheat sheet

- static_cast **between sibling bases is unsafe** unless you're certain of layout and relationship; **prefer** dynamic_cast **with** virtual base and **polymorphism**
- reinterpret_cast **should never be used for MI navigation** (non-portable, unsafe, does not account for subobject offsets)

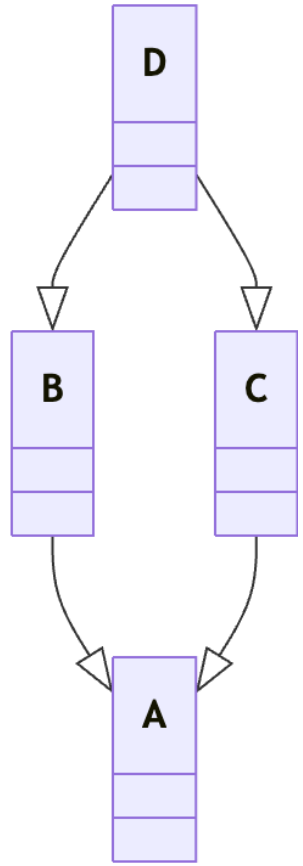
```
struct A { virtual ~A() = default; };
struct B { virtual ~B() = default; };
struct D : A, B {};

A* pa = new D;
// ⚠ This compiles only if you force it, but it's unsafe:
// ❌ Wrong: sibling cast; causes UB
B* pb1 = static_cast<B*>(pa);

// ✅ Safe
B* pb2 = dynamic_cast<B*>(pa);

// ❌ Non-portable/unsafe: does not account
//    for subobject offsets
B* pb2 = reinterpret_cast<B*>(pa);
```

OOP multiple inheritance: The Diamond Problem

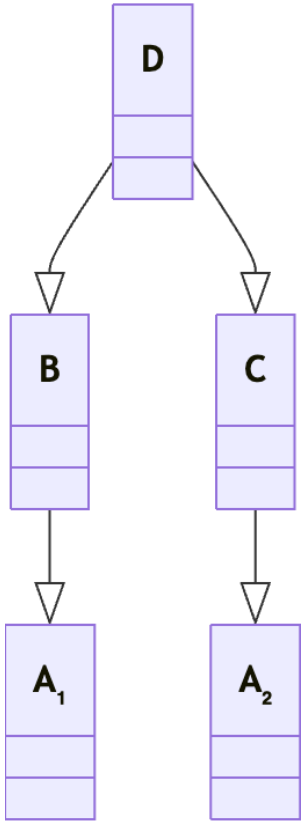
- This is the classic Multiple Inheritance pitfall, common to many languages
- If D **derives from two classes** (e.g., B and C) **that derive from** the same base, A
- This means that **it inherits two Base subobjects** (A_1 and A_2, via B and C respectively) causing:

1. Ambiguity converting D to Base (which one?)
2. State duplication in Base (inconsistent updates)

Class diagram of the Diamond Problem



Consequence: double base instance



C++ multiple inheritance: *virtual* derivation

- ***virtual* derivation** is a **mechanism to specify that only one copy of the base class should be created**
  - Only **one** Base subobject inside D
  - **Most derived** class (D) initializes the virtual base
- This solves duplication and conversion ambiguities
- Each intermediate class must use the *virtual* keyword

```
struct Base { int x; };

struct A : virtual Base { };
struct B : virtual Base { };

struct D : A, B {
   // single shared Base subobject
   D() {
      x = 42;
   }
};
```

C++ multiple inheritance: *virtual* derivation

- Normally a constructor initializes only its own variables and the base class
- **Virtually inherited base classes** are an exception: they **are initialized only once by the farthest class in the derivation tree**
- **Intermediate classes cannot** initialize the virtual base when constructing the most derived object
- If the virtual base has no default constructor, the most derived class **must** explicitly call it
- However, **virtual inheritance may introduce pointer adjustment** thunks* **and extra indirection**, but it's the **standard cure for diamonds**

```
struct Base { int x; };

struct A : virtual Base { };
struct B : virtual Base { };

struct D : A, B {
   // single shared Base subobject
   D() {
      x = 42;
   }
};
```

**a small piece of code that is called as a function*, does some small thing, and then JUMP s to another location

C++ multiple inheritance: *virtual* derivation

So Base is not initialized by A or B, but by D

A and B must initialize Base, but these operations are ignored if an object of type D is instantiated

```
struct Base {
    Base() { std::cout << "Base()\n"; }
    int x = 0;
    virtual ~Base() = default;
};
struct A : virtual Base {
    A() { std::cout << "A()\n"; }
    void setA() { x = 1; }
};
struct B : virtual Base {
    B() { std::cout << "B()\n"; }
    void setB() { x = 2; }
};
struct D : A, B {
    D() : Base(), A(), B() { std::cout << "D()\n"; }
    void print() const { std::cout << "x=" << x << "\n"; }
};
D d;
d.setA();
d.setB();
d.print(); // x=2 (same shared Base::x)
Base* pb = &d; // OK: single virtual base
B* pb_to_B = dynamic_cast<B*>(pb); // OK: polymorphic via Base's virtual dto
```

Source: stackoverflow.com

C++ multiple inheritance: *virtual* derivation

- **Common implementations** will **make access** to data members **of virtual base** classes **use an additional indirection**
- **Calling** a member **function of a base class in a multiple inheritance scenario will need adjustment of the *this* pointer**
- If that base class is virtual, then the offset of the base class sub-object in the derived class's object depends on the dynamic type of the derived class and will need to be calculated at runtime

Source: stackoverflow.com

C++ multiple inheritance: *virtual* derivation

- *Whether this has any **visible performance impact** on real-world applications **depends on many things**:*
- ***Do virtual bases have data members** at all? **Often, it's abstract base classes** that need to be derived from virtually, **and abstract bases that have any data members are often a code smell anyway***
- ***Assuming you have virtual bases with data members, are those accessed in a critical path**? If a user clicking on some button in a GUI results in a few dozen additional indirections, nobody will notice*
- ***What would be the alternative** if virtual bases are avoided? Not only **might the design be inferior**, **it is also likely that the alternative design has a performance impact, too**. It has to achieve the same goal, after all, and TANSTAAFL*. Then you traded one performance loss for another plus an inferior design.*

*There ain't no such thing as a free lunch

C++ classes: order of Construction / Destruction

- Construction order:
  1. Virtual bases (if any) (once), **before** anything else (top-most first)
  2. Non-virtual bases **left-to-right as** declared (e.g., class D : A, B)
  3. Members in declaration order
  4. Most-derived constructor body
- Destruction in **reverse order**

```
struct D : A, B {
  D() : Base(/*...*/), A(), B() { /*...*/ }
};
```

If A and B also try to initialize Base (virtual), those are ignored—only D's matters.

```
class Config {
public:
  static Config& instance() {
    static Config* p = new Config(/*... */);
    return *p; // never delete *p
  }
};
```

C++ classes: key takes on polymorphism

- **Public inheritance = is-a**; anything valid for Base must make semantic sense for Derived (LSP)
- **Use override/final for robustness**; re-expose hidden overloads with using
- ***dynamic_cast* for safe downcasts**; if you keep needing it, reconsider the interface and rely on virtual functions
- Multiple inheritance is fine for interfaces; use **virtual inheritance** to fix the diamond

Mutable

- ***mutable*** keyword **allows a non-static** data **member of a class to be modified even if the containing object is const**
- **Useful for logic** like caching, lazy evaluation, or internal counters **that don't affect logical constness**
- Applies only to **non-static data members and does not affect thread safety:** mutable members can still cause data races

```
struct Cache {
    mutable int hits = 0;
    int value = 42;

    int get() const {
        ++hits;       // allowed because hits is mutable
        return value; // value cannot be modified here
    }
};
```

C++ multiple inheritance: Practical Patterns

- **Non-virtual destructor in polymorphic base cause UB on delete:** always declare base destructor virtual or = default
- **Ambiguous calls (same name in multiple bases):** qualify explicitly (A::f()), or override in derived
- **State duplication in diamonds:** use **virtual inheritance** to share a single base subobject
- **Hidden overloads / name hiding:** use using Base::f; to re-expose overloads
- **Fragile hierarchies with heavy implementation MI:** prefer **interfaces + composition**, or **mixins/CRTP** for orthogonal behaviors